

VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



WEB APPLICATION DEVELOPMENT
IT093IU

FINAL REPORT

Topic: Social Media Web Application

By Group VietnamSocial – Member List

1	Nguyễn Thành Nam	ITCSIU22311	Backend + Report
2	Hồ Thiên Trường	ITCSIU22306	Front-end

Instructor: MSc. Nguyen Trung Nghia

Link GitHub: <https://github.com/TruongHo22306/WebApplicationDev>

TABLE OF CONTENTS

LIST OF FIGURES	v
LIST OF TABLES	vi
ABSTRACT	vii
CHAPTER 1	1
INTRODUCTION	1
1.1. Problem Statement	1
1.2. Scope and Object	1
1.2.1. Scope	1
1.2.2. Objectives	2
1.3. Report Structure	2
CHAPTER 2	3
METHODOLOGY	3
2.1. Overview	3
2.2. User Requirement Analysis	3
2.2.1. Requirement Register (Functional and Non-functional).....	3
2.2.2. Risk register	4
2.3. Requirement Specifications	6
2.3.1. Use Case Diagram	6
2.3.2. Use Case Descriptions	7
2.4. System Design	11
2.4.1. System Architecture.....	11
2.4.2. Table Schema	12
2.4.3. Class Diagram (High-Level)	14
2.4.4. Sequence Diagram	15
2.4.4.1. Create Post	15
2.4.4.2. Advance Story Creation	16
CHAPTER 3	17
IMPLEMENTATION & RESULTS	17
3.1. Implement	17
3.1.1. Front-end.....	17

3.1.2.	Back-end	17
3.1.3.	Database	17
3.1.4.	Third-Party Integration (Media Infrastructure)	17
3.2.	Results.....	18
CHAPTER 4		20
DISCUSSION.....		20
4.1.	Development Challenges and Iterations	20
4.2.	System Evaluation and Testing	20
CHAPTER 5		21
CONCLUSIONS AND FEATURE WORKS.....		21
5.1	Conclusions.....	21
5.2	Future works	21
REFERENCES		22

LIST OF FIGURES

<i>Figure 2.1: Use case diagram</i>	<i>6</i>
<i>Figure 2.2: MVC</i>	<i>11</i>
<i>Figure 2.3: Table Schema</i>	<i>12</i>
<i>Figure 2.4: Class Diagram.....</i>	<i>14</i>
<i>Figure 2.5: Sequence Diagram_ Create Post.....</i>	<i>15</i>
<i>Figure 2.6: Sequence Diagram _Advance Story Creation</i>	<i>16</i>
<i>Figure 3.1: Log in page.....</i>	<i>18</i>
<i>Figure 3.2: Register info</i>	<i>18</i>
<i>Figure 3.3: Home Feed.....</i>	<i>18</i>
<i>Figure 3.4: Story.....</i>	<i>18</i>
<i>Figure 3.5: Create Story.....</i>	<i>18</i>
<i>Figure 3.6: Create post.....</i>	<i>18</i>
<i>Figure 3.7 : Chat interface.....</i>	<i>18</i>
<i>Figure 3.8: Setting.....</i>	<i>18</i>
<i>Figure 3.9: People's Profile</i>	<i>19</i>

LIST OF TABLES

<i>Table 2.1: Functional requirements</i>	<i>3</i>
<i>Table 2.2: Non-functional requirements</i>	<i>4</i>
<i>Table 2.3: Risk Register.....</i>	<i>4</i>
<i>Table 2.4: UC1_Sign Up.....</i>	<i>7</i>
<i>Table 2.5: UC2_Log In.....</i>	<i>8</i>
<i>Table 2.6: UC3_View 3D Jewelry</i>	<i>9</i>
<i>Table 2.7: UC4_Like a Post.....</i>	<i>10</i>
<i>Table 2.8: User Collection.....</i>	<i>12</i>
<i>Table 2.9: Post Collection.....</i>	<i>13</i>
<i>Table 2.10: Story Collection.....</i>	<i>13</i>

ABSTRACT

This report details the design and development of **Mellow**, a Social Media Web Application designed to enhance digital engagement by prioritizing technical security and a high-fidelity user experience. The proposed Minimum Viable Product (MVP) integrates core social functionalities, including User Authentication, Content Feed Management, and social interactions, within a robust architectural framework. The project stands out with its advanced content creation tools, especially the in-app Story Editor that offers professional creative control. Its hybrid system balances speed and security, using middleware authorization and JSON Web Tokens (JWT) for protection. A Cloud-based Content Delivery Network (CDN) ensures fast media processing. By integrating user-friendly editing with a secure backend, Mellow creates a smooth and engaging environment for social interaction.

CHAPTER 1

INTRODUCTION

1.1. Problem Statement

Despite the widespread use of social media platforms, users often encounter significant challenges that reduce both engagement and satisfaction. A primary issue is content fatigue, driven by a platform's inability to facilitate the creation of high-quality, personalized media. This is compounded by the absence of robust, professional-level creative tools within native applications, which restricts a user's capacity to produce engaging content. Consequently, this creates a friction-filled environment that slows the core cycle of content creation, sharing, and interaction. This project addresses these challenges by introducing advanced features, including an integrated multi-layered media editor and optimized cloud-based delivery designed to maximize creativity and revitalize user engagement.

1.2. Scope and Object

1.2.1. Scope

The application encompasses the robust foundation necessary for a modern social media platform, alongside specialized technical implementations designed to enhance user creativity and platform performance.

Core Platform Functionalities:

- **User Management (Authentication & Authorization):** Secure registration and login/logout utilizing JSON Web Tokens (JWT) and middleware-level route protection.
- **Content/Feed Management:** Mechanisms to create, store, and view multi-format posts (text and images) and a dynamic main user feed.
- **Interaction & Social Connection:** Features for user engagement, including Like/Unlike toggles, and social connection tools such as search and follow/unfollow functionality.

Advanced Technical Focus Areas:

- **Advanced Content Generation:** Integration of professional-grade, in-app editing tools for stories, supporting multi-layered content including stickers, text manipulation, and coordinate-based drawing paths.
- **High-Performance Media Infrastructure:** Implementation of a hybrid storage architecture that utilizes a Cloud-based Content Delivery Network (CDN) to replace heavy data-URI transfers, ensuring sub-second media loading and optimized server payloads

1.2.2. Objectives

Our project aims to create an intuitive and efficient social media platform by achieving the following objectives:

- **User Empowerment through Creative Tools:** Provide an intuitive platform for generating high-quality content using in-app editing tools, simplifying the creative process for all users.
- **Performance Optimization (Efficiency):** Minimize content delivery latency by implementing a hybrid cloud storage architecture, replacing legacy Base64 data transfers with optimized CDN-delivered media.
- **Data Integrity and Security:** Implement robust measures to protect user data through secure JSON Web Token (JWT) authentication and middleware-level authorization, preventing unauthorized access.
- **Responsive Accessibility:** Ensure a seamless user-friendly experience across various devices with a responsive UI that provides real-time feedback and state persistence

1.3. Report Structure

This report is organized into six chapters. Chapter 1 introduces the background, problem statement, scope and objectives, and provides an overview of the report structure. Chapter 2 describes the methodology adopted, including requirement analysis, system design, and specification details. Chapter 3 presents the product result. Chapter 4 discusses the overall development, testing strategies, comparison with initial objectives, and evaluation of system performance. Finally, Chapter 5 concludes the report and suggests potential directions for future improvements.

CHAPTER 2

METHODOLOGY

2.1. Overview

This chapter explains the development methodology for the **Mellow Social Media System**. It begins with an analysis of user needs using Requirement and Risk Registers, followed by the presentation of system functionalities through Use Case Diagrams. The **System Architecture** is then introduced to illustrate the high-level integration between the frontend, backend, and Cloud-native media services. Finally, the platform's structure and operational behavior are detailed through Schema, Class, and Sequence Diagrams, with a specific focus on the secure data flow required for real-time content editing and optimized media delivery.

2.2. User Requirement Analysis

2.2.1. Requirement Register (Functional and Non-functional)

Table 2.1: Functional requirements

Req.ID	Requirement Name	Detailed Description
01	User Authentication	Users must be able to register, log in, and log out securely using email and password, protected by JWT.
02	Media Upload	Users must be able to upload images for posts and stories directly from their local device to cloud storage.
03	Advanced Story Editing	Users must be able to add, resize, and position stickers, text, and freehand drawings over story media.
04	Dynamic Social Feed	The system must display a chronological feed of posts and a horizontal panel for active user stories.
05	Social Interaction	Users must be able to like or unlike posts with immediate visual feedback in the UI.
06	Reel/Video Viewing	The system must support vertical video playback with auto-play and scroll-to-view functionality.
07	Performance (UX)	Media assets (images/videos) must load within 2 seconds using a Content Delivery Network (CDN) to minimize latency.

Table 2.2: Non-functional requirements

Req.ID	Requirement Name	Detailed Description
01	Security (Data)	Private routes and sensitive actions (delete/post) must require a valid JWT for authorization.
02	Scalability	The storage architecture must handle large media files without impacting the speed of the primary database queries.
03	Usability	The content creation interface must provide real-time visual previews of edits before final submission.
04	Responsiveness	The application layout must adapt seamlessly across desktop, tablet, and mobile screen dimensions.

2.2.2. Risk register

Table 2.3: Risk Register

Risk ID	Risk Category	Risk Title	Affect	Prob.	Impact	Risk Response Plan
01	Technical	Data Payload Overhead	Large Base64 strings causing server timeouts or database bloat.	High	High	Implemented Cloudinary CDN to offload media storage and optimized Multer to handle binary streams instead of text.
02	Security	Unauthorized Data Access	Malicious users attempting to delete or modify posts they do not own.	Medium	High	Implemented JWT-based protect middlewares to verify user identity on every sensitive backend request.

03	Performance	High Latency	Slow content loading times reducing user retention and engagement.	Medium	High	Switched to a distributed Cloud storage model (Cloudinary) to ensure sub-second image delivery through edge locations.
04	Financial	API Quota Exhaustion	Exceeding Cloudinary free-tier limits due to high volume of high-resolution uploads.	Low	Medium	Implement client-side image compression and set strict Multer limits to prevent massive file uploads.
05	Stability	Third-party Service Outage	App failure if Cloudinary or MongoDB Atlas goes offline.	Low	High	Implement robust error handling in the frontend and backend to show friendly "Service Unavailable" messages rather than crashing.

2.3. Requirement Specifications

2.3.1. Use Case Diagram

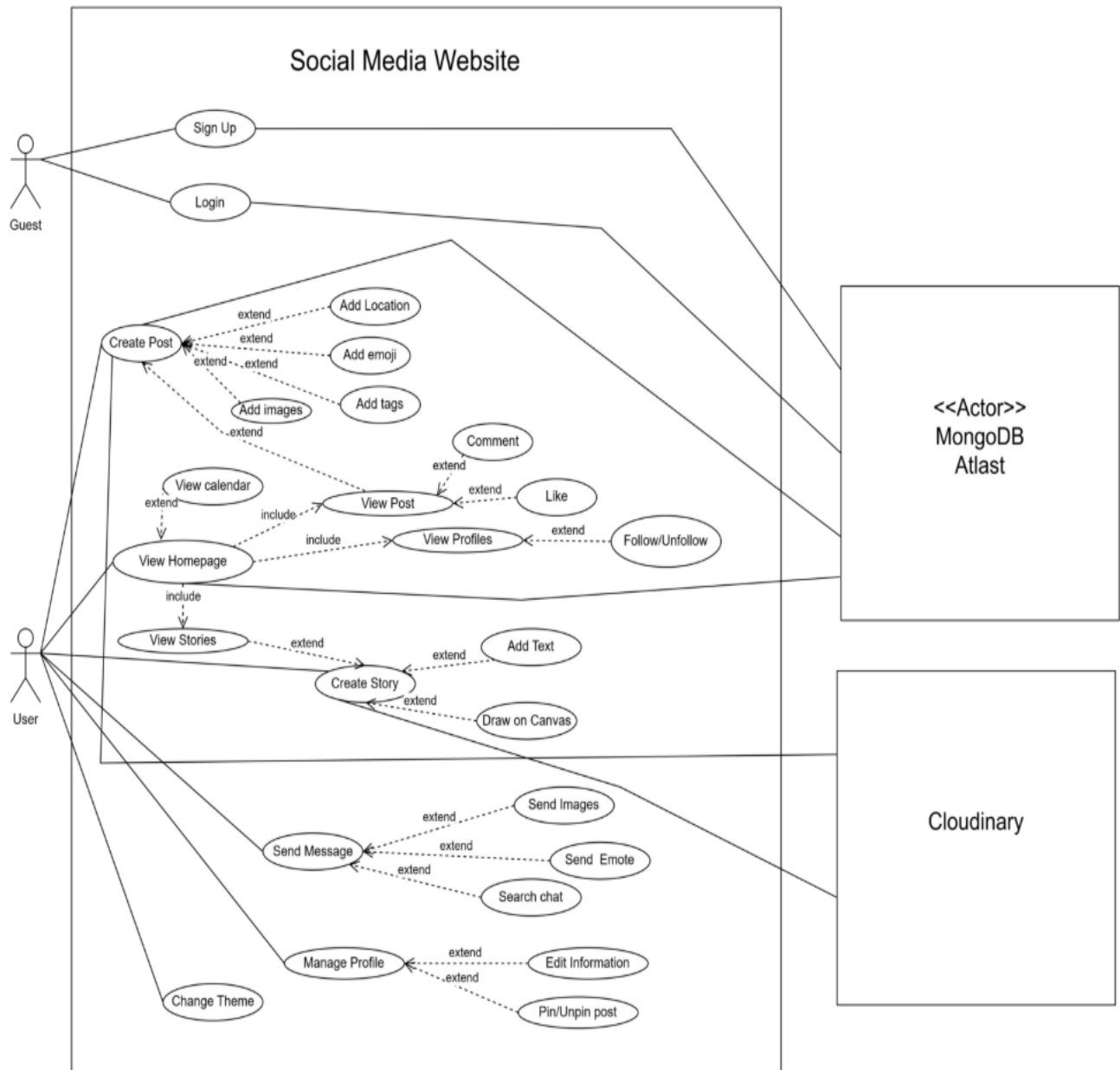


Figure 2.1: Use case diagram

2.3.2. Use Case Descriptions

0.1 Use Case 1: Sign Up

Name: Sign Up

Identifier: UC1

Inputs:

1. Personal information (First Name, Last Name, Email, Phone Number)
2. Account credentials (Password, Confirm Password)

Outputs:

1. Confirmation message [If successful]
2. Error message [If failed]

Basic Course:

Table 2.4: UC1_Sign Up

Actor: Guest	System
Access the registration page.	Display registration form.
Enter personal information and account credentials.	Real-time validation of fields (e.g., password length).
Submit the registration form.	Validate the provided information. Store the user credentials and personal info. Display success and redirect to Login.

Preconditions:

1. The user has not created an account yet.

Postconditions:

1. The user's account is created and stored in the system.

User Story:

As a user, I want to create an account so that I can interact with other members on the platform.

0.2 Use Case 2: Create Post

Name: Create Post

Identifier: UC2

Inputs:

1. Text content (Caption)
2. Media attachments (Images from local storage)
3. Optional - Metadata (Hashtags, Location)

Outputs:

1. Post added to feed [If successful]
2. Error message [If failed]

Basic Course:

Table 2.5: UC2_Log In

Actor: Member	System
Selects "New Post" from the navigation.	Displays the post creation interface.
Uploads an image file.	Processes file via Multer and generates a preview.
Enters caption, hashtags, and location.	Validates character limits and tag counts.
Submits the post.	Streams image to Cloudinary and saves URL/metadata to MongoDB . Displays success message and redirects to Home.

Preconditions:

1. The user is logged in and authenticated via JWT.

Postconditions:

1. The media is stored in the CDN and the post metadata is live in the database.

User Story:

As a user, I want to share my photos with a caption so that my followers can see my updates in their feed.

0.3 Use Case 3: Advanced Story Creation

Name: Advanced Story Creation

Identifier: UC3

Inputs:

1. Background media (Gradient or Uploaded Image)
2. Interactive objects (Stickers, Text)
3. Drawing data (Freehand strokes)

Outputs:

1. Story published to Story Panel [If successful]
2. Error notification [If failed]

Basic Course:

Table 2.6: UC3_View 3D Jewelry

Actor: Guest	System
Accesses the Story Creation tool.	Displays the interactive canvas.
Chooses a background or uploads an image.	Renders the selected background on the canvas.
Adds stickers, text, or freehand drawings.	Tracks and displays object coordinates and stroke paths in real-time.
Finalizes and publishes the story.	Converts canvas data to coordinate-based JSON and uploads any images to Cloudinary. Stores the complex JSON object in MongoDB and updates the Story Panel.

Preconditions:

1. The user is authenticated.
2. The system is connected to Cloudinary and MongoDB Atlas

Postconditions:

1. A temporary multi-layered story is created and associated with the user profile.

User Story:

As a creator, I want to use professional-grade editing tools like stickers and drawing so that I can produce more personalized and engaging content for my audience.

0.4 Use Case 4: View and Like a Post

Name: View and like a Post

Identifier: UC4

Inputs:

1. Click interaction on the "Like" icon

Outputs:

1. Updated like count and active state for the icon

Basic Course:

Table 2.7: UC4_Like a Post

Actor: Member	System
Scroll through Feed, Profile, or Reels.	Fetches post metadata from MongoDB and media from Cloudinary.
Views a post and clicks the heart icon.	Sends an update request with the User ID and Post ID. Toggles the like status in the database and returns the new like count. Updates the UI heart color and count immediately (Optimistic UI).

Preconditions:

1. The user must be logged in and viewing a post.

Postconditions:

1. The engagement metric for the post is permanently updated in the database.

User Story:

As a user, I want to like posts so that I can show support for content I enjoy and influence the feed's engagement metrics.

2.4. System Design

2.4.1. System Architecture

The social media web application is designed using a three-tier architecture, consisting of the frontend, backend, and a hybrid data storage layer. This architecture promotes separation of concerns, scalability, and ease of maintenance. The backend follows the Model–View–Controller (MVC) pattern and communicates with the frontend through RESTful APIs.

Model–View–Controller (MVC): This pattern structures the backend logic to separate data management from business processing.

- **Model:** Responsible for managing data schemas (e.g., Users, Posts, Stories) within MongoDB Atlas.
- **Controller:** Implements the core business logic, such as JWT-based authentication and the processing of binary media streams using Multer middleware.
- **View:** Represented by the React.js frontend, which provides the user interface and captures complex user interactions like real-time story editing.

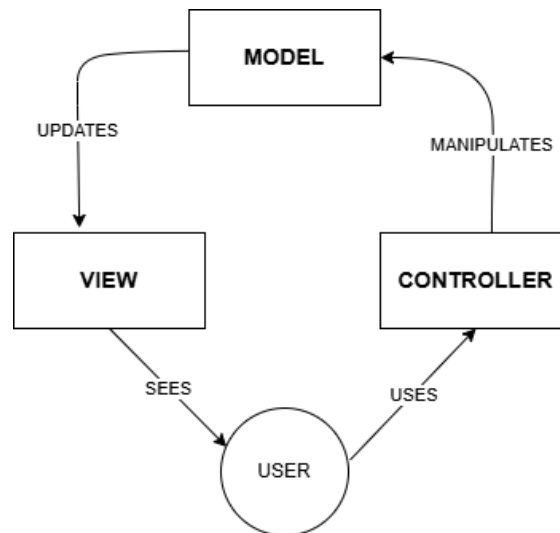


Figure 2.2: MVC

REST (Representational State Transfer): RESTful APIs enable communication between the frontend and backend through standard HTTP methods. This approach is chosen because REST is stateless and scalable. To overcome the performance limitations of loading complex social media feeds, the system integrates a **Cloud-native CDN (Cloudinary)**. This ensures that while metadata is fetched via REST, heavy media assets are delivered through optimized edge locations, significantly reducing latency.

2.4.2. Table Schema

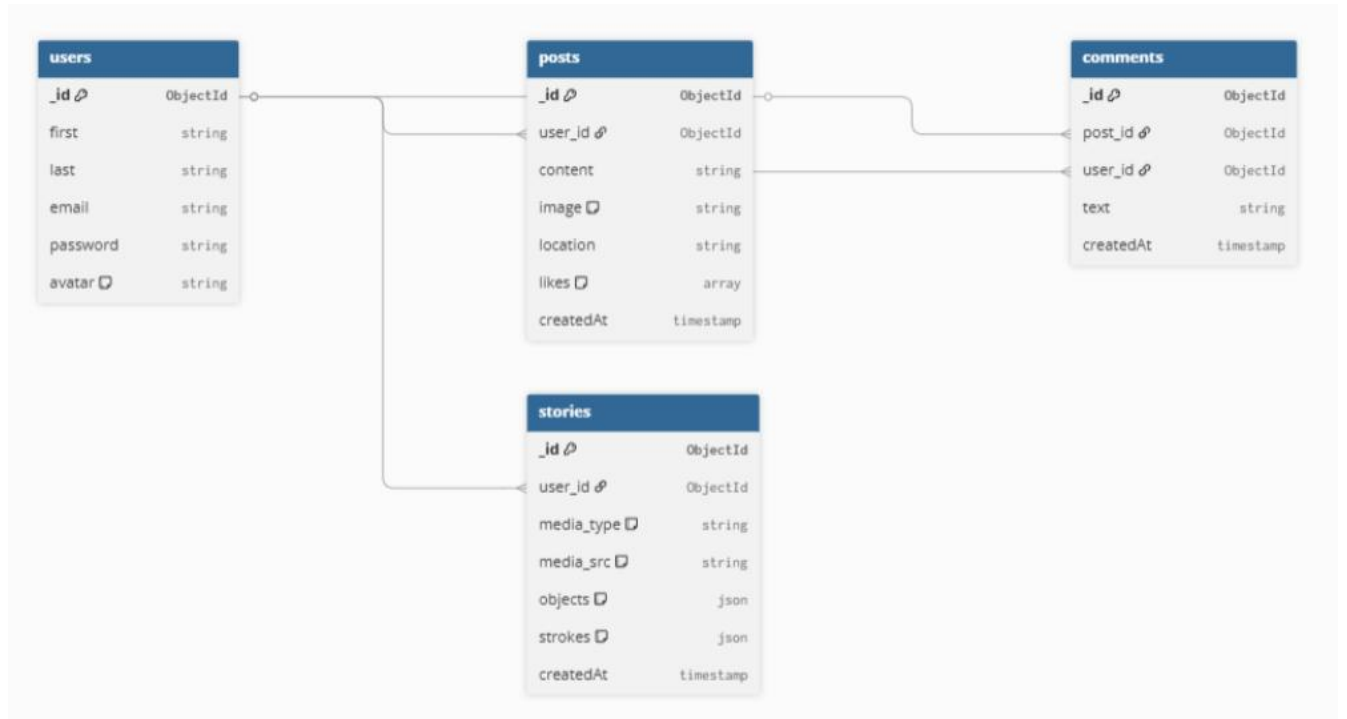


Figure 2.3: Table Schema

Schema description:

Table 2.8: User Collection

Field Name	Data Type	Constraints	Description
<code>_id</code>	ObjectId	Primary Key	Unique identifier for the user.
<code>first</code>	String	Required	User's first name.
<code>last</code>	String	Required	User's last name.
<code>email</code>	String	Unique, Required	User's login email.
<code>password</code>	String	Required	Hashed password for security.
<code>avatar</code>	String	Optional	URL to the profile image stored in Cloudinary.

Table 2.9: Post Collection

Field Name	Data Type	Constraints	Description
_id	ObjectId	Primary Key	Unique identifier for the post.
user	ObjectId	Ref: 'User'	Reference to the user who created the post.
content	String	Required	The text/caption of the post.
image	String	Required	Secure URL of the image hosted on Cloudinary.
location	String	Optional	Geographic location text.
likes	Array	[ObjectId]	Array of User IDs who liked the post.
createdAt	Date	Default: Now	Timestamp for chronological sorting.

Table 2.10: Story Collection

Field Name	Data Type	Constraints	Description
_id	ObjectId	Primary Key	Unique identifier for the post.
user	ObjectId	Ref: 'User'	Reference to the user who created the post.
content	String	Required	The text/caption of the post.
image	String	Required	Secure URL of the image hosted on Cloudinary.
location	String	Optional	Geographic location text.
likes	Array	[ObjectId]	Array of User IDs who liked the post.
createdAt	Date	Default: Now	Timestamp for chronological sorting.

2.4.3. Class Diagram (High-Level)

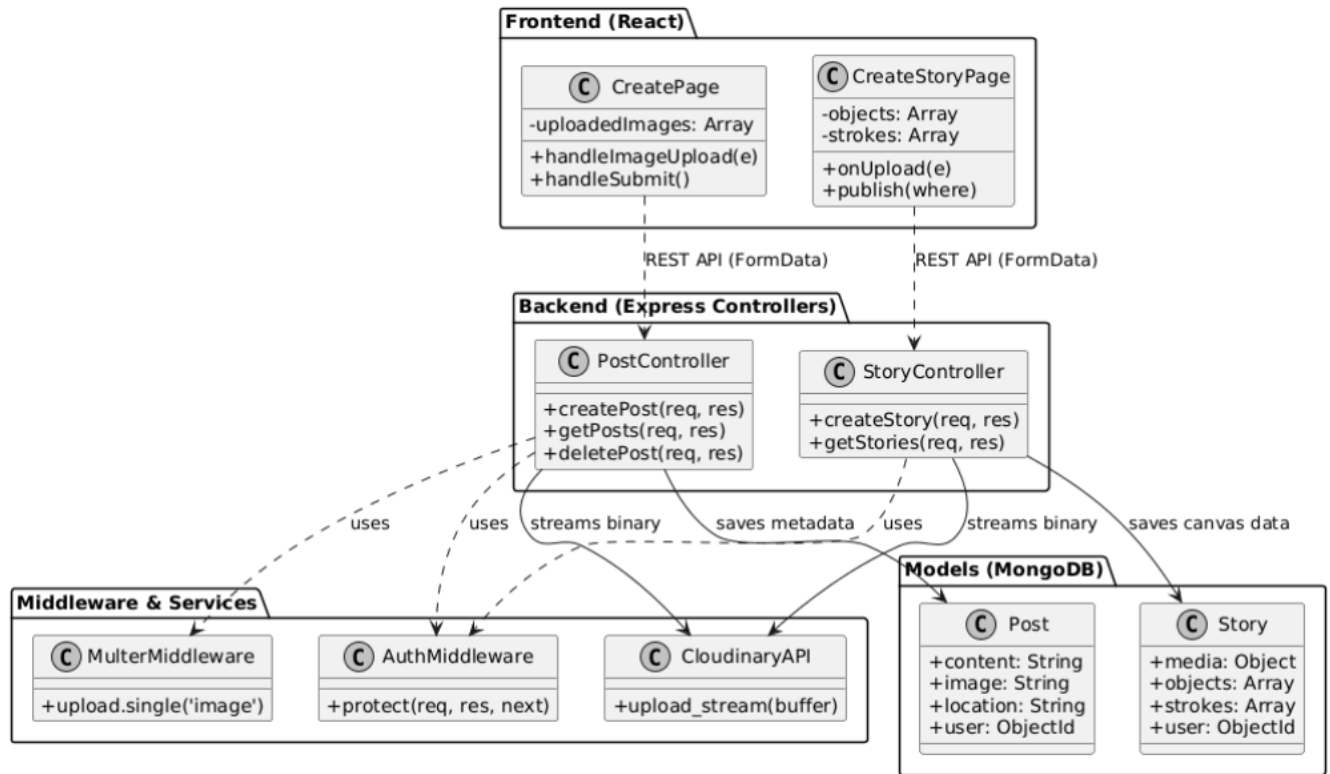


Figure 2.4: Class Diagram

Explanation:

1. Boundary Classes (Frontend Layer)

- `CreatePage` & `CreateStoryPage`: These React components serve as the user interface layer.
- They are responsible for state management (e.g., tracking hashtags and canvas strokes) and packaging user data into `FormData` for transmission to the server.

2. Controller & Middleware Classes (Backend Logic)

- `PostController` & `StoryController`: These classes handle the business logic. They receive API requests, process data, and coordinate with external services.
- `AuthMiddleware`: A security layer that utilizes JWT (JSON Web Tokens) to protect private routes, ensuring only authenticated users can execute "Create" or "Delete" actions.
- `MulterMiddleware`: A specialized utility that intercepts multi-part form data, extracting binary media buffers without saving them to local disk.

3. Service Classes (External Integration)

- `CloudinaryAPI`: This external service class represents the system's dependency on cloud-native media storage.

- It receives binary streams from the controllers and returns a `secure_url`, which is then used as the permanent reference for images and videos.

4. Entity Classes (Data Models)

- User, Post, and Story Models: These represent the Mongoose Schemas that define the structure of the MongoDB documents.
- The relationship between User and Post/Story is a "One-to-Many" association, where each content piece stores a reference to its creator's Unique ID.

2.4.4. Sequence Diagram

2.4.4.1. Create Post

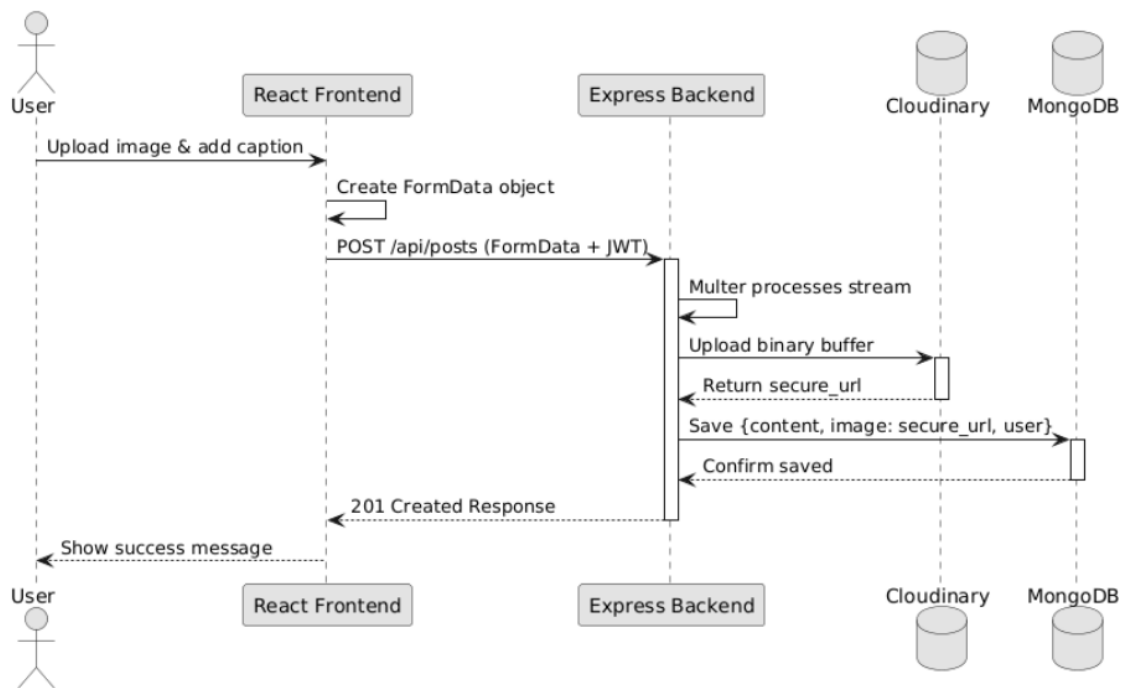


Figure 2.5: Sequence Diagram _Create Post

Description :

- **User Action:** The User selects an image and enters a caption in the **React Frontend**.
- **Data Packaging:** The frontend bundles the inputs into a FormData object for multipart transmission.
- **Streaming:** The **Express Backend** uses **Multer** to intercept the image buffer in memory, bypassing local disk storage.
- **Cloud Offloading:** The backend streams the buffer to **Cloudinary** and receives a `secure_url`.
- **Persistence:** The backend saves the caption and the Cloudinary URL to **MongoDB**, keeping the database record lightweight.

2.4.4.2.Advance Story Creation

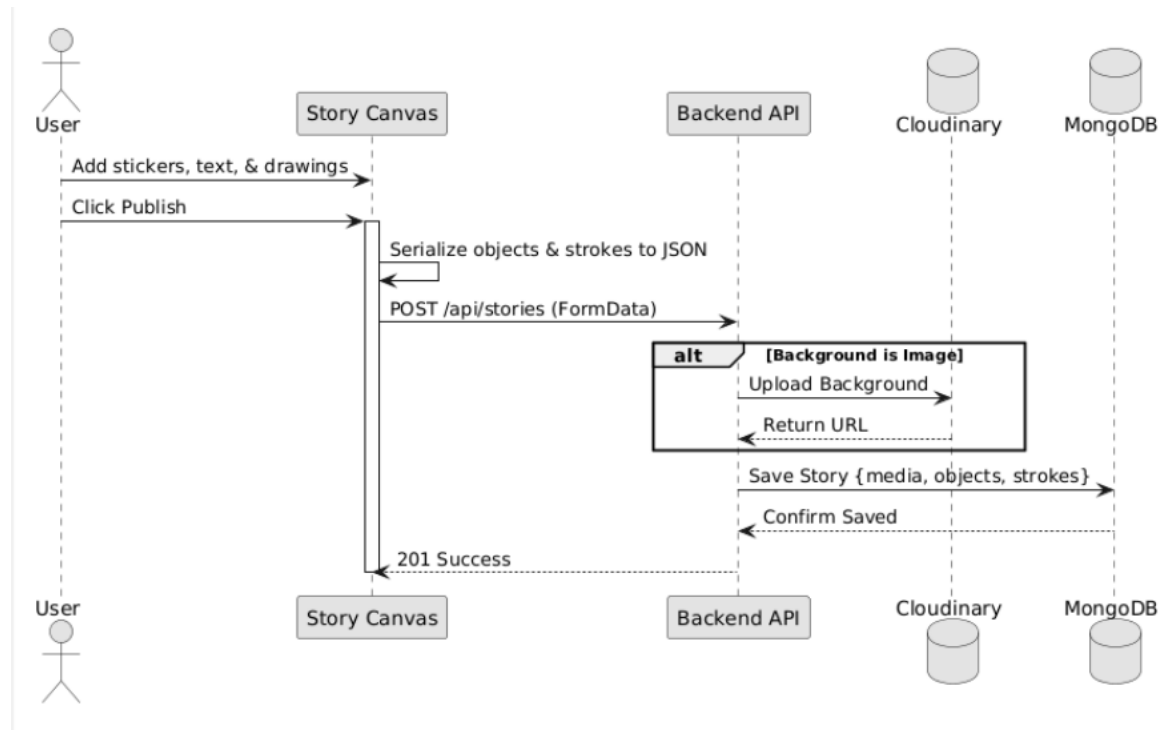


Figure 2.6: Sequence Diagram _Advance Story Creation

Description

- **Canvas Interaction:** The User adds stickers, text, and drawings to the **Story Canvas**.
- **Serialization:** Upon publishing, the frontend converts all visual objects into a coordinate-based JSON array.
- **Security:** The **AuthMiddleware** verifies the user's JWT before the request reaches the controller.
- **Hybrid Save:** The backend uploads any background images to **Cloudinary** while saving the coordinate JSON directly to **MongoDB**.
- **Reconstruction:** The saved state allows the system to perfectly redraw the story for other users without storing heavy flattened images.

CHAPTER 3

IMPLEMENTATION & RESULTS

3.1. Implement

3.1.1. Front-end

The frontend of the system is developed using HTML, CSS, and JavaScript, with **React.js** serving as the primary framework. React.js was chosen for its component-based architecture, which enabled the creation of reusable UI elements like the dynamic story panel and the interactive post feed. The framework's ability to handle complex state transitions—such as tracking sticker coordinates and freehand drawing paths in the Story Editor—was critical for maintaining a high-quality user experience. While React provides efficient UI updates, the state management required for "Optimistic UI" updates (e.g., immediate like-count changes) required careful implementation to ensure data consistency with the backend.

3.1.2. Back-end

The backend is built using **Node.js** and **Express.js**. This environment was selected to manage the high concurrency typical of social media platforms. A key implementation detail is the integration of **Multer middleware**, which allows the system to handle multipart form data. This was essential for transitioning away from heavy Base64 data transfers, which previously caused "SyntaxErrors" and server timeouts. By using stream-based processing, the backend can now intercept binary media files and transmit them to external storage without exhausting server memory.

3.1.3. Database

Database MongoDB Atlas serves as the primary data storage solution. Its document-based model is ideal for storing the complex JSON arrays required for the Story feature, such as the objects and strokes collections. To optimize performance and security, the database is used to store lightweight metadata and secure URLs rather than raw binary image data. This hybrid approach ensures that the database remains scalable and avoids the performance degradation associated with storing large BLOBs (Binary Large Objects).

3.1.4. Third-Party Integration (Media Infrastructure)

To address the critical requirement of performance, **Cloudinary** was integrated as a cloud-native Content Delivery Network (CDN). The backend utilizes the Cloudinary API to offload the storage and delivery of high-resolution images and videos. This integration was a primary solution for reducing latency, ensuring that media assets are served via optimized global edge locations, which significantly improved the overall responsiveness of the "Reels" and "Stories" features.

3.2. Results

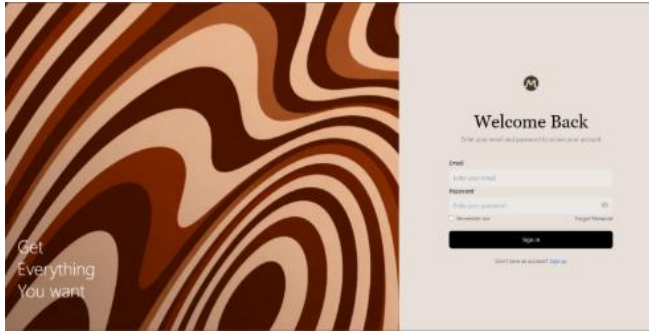


Figure 3.1: Log in page



Figure 3.2: Register info

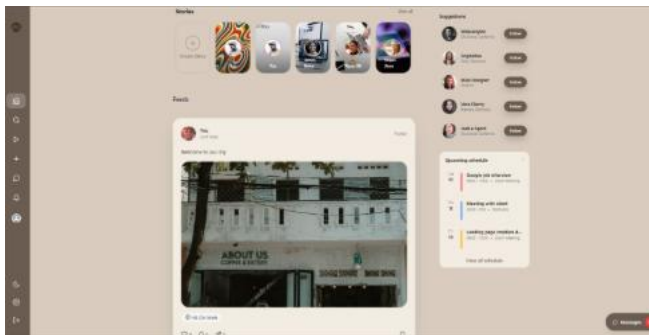


Figure 3.3: Home Feed

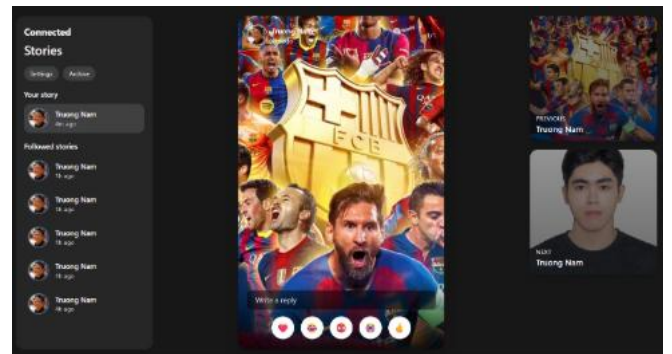


Figure 3.4: Story

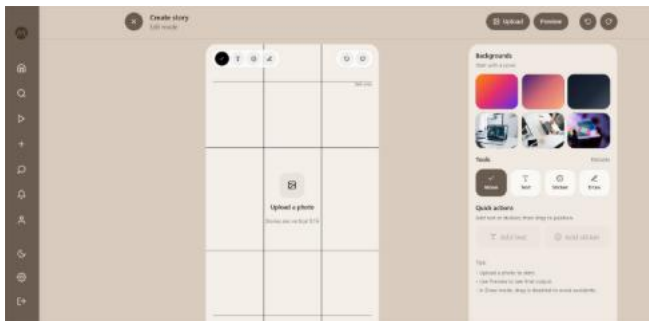


Figure 3.5: Create Story

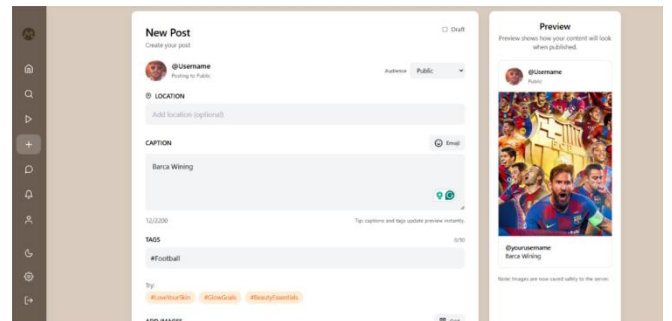


Figure 3.6: Create post

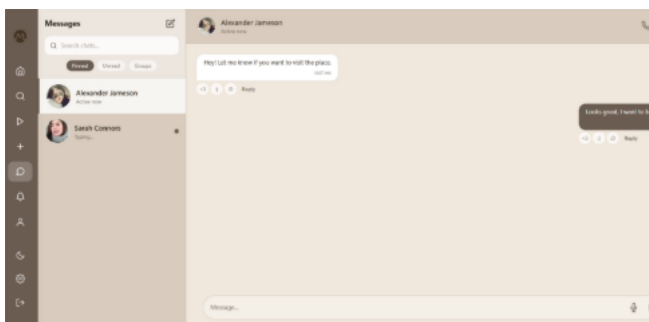


Figure 3.7: Chat interface



Figure 3.8: Setting

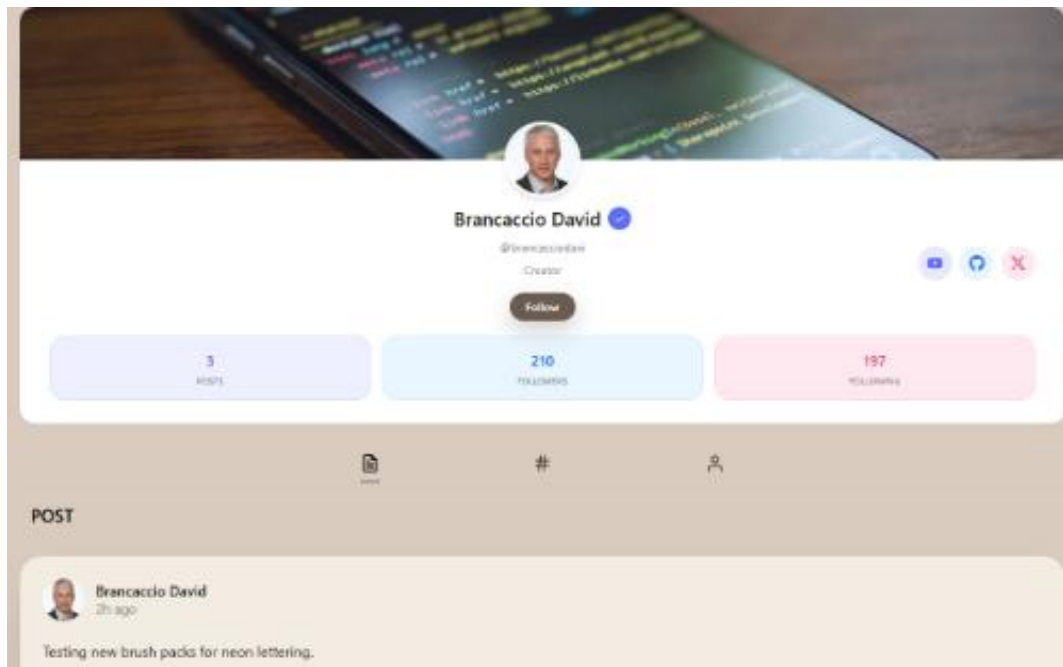


Figure 3.9: People's Profile

CHAPTER 4

DISCUSSION

4.1. Development Challenges and Iterations

The development process involved several critical transitions, most notably the shift in media handling strategy.

- **Transition from Base64 to Binary Streaming:** Initially, the system attempted to store images as Base64 strings directly in MongoDB, which led to "SyntaxError: Unexpected token" and server crashes due to the massive payload size.
- **Integration of Multer and Cloudinary:** To resolve these performance issues, the architecture was modified to use multer for in-memory buffering and the Cloudinary API for binary streaming, which stabilized the backend and reduced content loading times.
- **Canvas State Persistence:** Implementing the Story Editor required a complex coordination of drawing strokes and sticker coordinates.
- **Responsive Logic:** A significant challenge was ensuring that drawings made on one screen size appeared correctly on others, which was solved by storing relative coordinates in the database.

4.2. System Evaluation and Testing

The application underwent rigorous testing across both layers:

- **UI/UX Testing:** The frontend was tested for responsiveness across mobile and desktop views, ensuring the Story Canvas remained interactive and that the "Optimistic UI" for likes provided immediate feedback.
- **Backend and Security Testing:** The API endpoints were tested using tools like Postman to verify that the protect middleware correctly blocked unauthorized access to private data.
- **Media Pipeline Validation:** We verified that high-resolution images were successfully offloaded to Cloudinary and that the resulting secure URLs were correctly indexed in MongoDB.

CHAPTER 5

CONCLUSIONS AND FEATURE WORKS

5.1 Conclusions

This project successfully addressed the problem of low user engagement by providing professional-grade creative tools within a social media environment. By building an integrated Story Editor and a high-performance media infrastructure, the platform enables a seamless cycle of content creation and interaction. The move to a cloud-native architecture ensured that the application remains scalable and efficient, meeting the objectives of security, accessibility, and high performance defined at the start of the project.

5.2 Future works

While the core foundation is complete, the following features are planned for future iterations to further enhance the platform:

- **Reels/Short-form Video:** Implementing the "Create Reel" feature using the established Cloudinary pipeline to support vertical video uploads and auto-playing feeds.
- **Real-time Messaging:** Integrating a "Send Message" feature with WebSockets (Socket.io) to allow users to interact instantly beyond posts and comments.
- **Advanced Content Moderation:** Adding automated AI-based checks for image uploads to ensure community safety.
- **Enhanced Discovery:** Developing an algorithm-based search to help users find trending hashtags and personalized content more effectively.

REFERENCES

1. JON DUCKETT, "HTML and CSS: Design and Build Websites," 2011.
2. JON DUCKETT, "JavaScript and jQuery: Interactive Front-End Web Development," 2014.
3. ALEX BANKS, EVE PORCELLO, "Learning React: Modern Patterns for Developing React Apps," 2020.
4. KINSTA®, "What Is Express.js? Everything You Should Know," n.d.
5. ETHAN BROWN, "Learning Node.js Development," 2019.
6. GILLIS, A., "What is REST API (RESTful API)," 2024.